

CSS Container Queries

Complete Guide: Component-Based Responsive Design

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What are Container Queries?	2
1.2	Why Container Queries Matter	2
1.2.1	Real-World Applications	2
1.2.2	Benefits over Media Queries	3
1.3	Browser Support	3
2	Core Concepts	4
2.1	Container-Type Property	4
2.1.1	Syntax	4
2.1.2	Value Meanings	4
2.2	Container-Name Property	4
2.2.1	Syntax	4
2.3	Container Query Syntax	5
2.3.1	Basic Query	5
3	Practical Examples	6
3.1	Example 1: Responsive Card Component	6
3.2	Example 2: Typography Scaling	7
3.3	Example 3: Sidebar Responsive Layout	8
3.4	Example 4: Grid Gallery	8
3.5	Example 5: Named Multiple Containers	9
4	Advanced Techniques	11
4.1	Container Query Units	11
4.1.1	Container-Relative Units	11
4.2	Nested Containers	11
4.3	Style-Based Queries	12
5	Browser Support & Fallbacks	13
5.1	Progressive Enhancement	13
5.2	Feature Detection	13
6	Best Practices	14
6.1	Key Recommendations	14
6.1.1	1. Use Named Containers	14
6.1.2	2. Nest Containers Wisely	14
6.1.3	3. Test Multiple Contexts	15

6.2	Common Patterns	15
7	Common Issues & Solutions	16
7.1	Issue 1: Container Query Not Triggering	16
7.2	Issue 2: Layout Shift	16
8	Summary & Quick Reference	18
8.1	When to Use Container Queries	18
8.2	Quick Reference	18
8.2.1	Common Patterns	18
8.2.2	Complete Component Example	18

Chapter 1

Introduction

1.1 What are Container Queries?

Container queries enable components to respond to their container's size rather than the viewport size. This revolutionary CSS feature changes how we approach responsive design, shifting from viewport-based to component-based styling. With container queries, a component can adapt its layout based on the width, height, or style of its parent container.

```
/* Define container context */
.card-container {
  container-type: inline-size;
  container-name: card;
}

/* Query container size */
@container (min-width: 400px) {
  .card { display: grid; }
}

/* Or use named container */
@container card (min-width: 400px) {
  .card { display: flex; }
}
```

Container queries revolutionize component design, enabling truly reusable components that work anywhere without viewport-based breakpoints.

1.2 Why Container Queries Matter

1.2.1 Real-World Applications

- **Reusable Components:** Components adapt to any container
- **Design Systems:** Build flexible component libraries
- **Sidebar Layouts:** Respond to sidebar width changes
- **Modal Dialogs:** Adapt to dialog size, not viewport
- **Grid Galleries:** Columns adapt to parent container

- **Card Designs:** Same card works in different layouts

1.2.2 Benefits over Media Queries

- Components truly decoupled from viewport
- Same component works in multiple contexts
- Reduced CSS breakpoint complexity
- Better component encapsulation
- Dynamic layout adaptation
- Future-proof responsive design

1.3 Browser Support

Container queries are well-supported in modern browsers:

Browser	Version	Status
Chrome/Edge	v105+	Full Support
Firefox	v110+	Full Support
Safari	v16+	Full Support
Opera	v91+	Full Support
IE 11	N/A	Not Supported

Chapter 2

Core Concepts

2.1 Container-Type Property

The `container-type` property establishes a container query context.

2.1.1 Syntax

```
/* Track inline (width) size */
container-type: inline-size;

/* Track block (height) size */
container-type: block-size;

/* Track both dimensions */
container-type: size;

/* No container context */
container-type: normal;
```

2.1.2 Value Meanings

Value	Meaning
inline-size	Query width dimension (most common)
block-size	Query height dimension
size	Query both dimensions
normal	No container context

2.2 Container-Name Property

The `container-name` property names containers for specific targeting.

2.2.1 Syntax

```
/* Unnamed container */
container-type: inline-size;
```

```
/* Named container */
container-name: card;
container-type: inline-size;

/* Shorthand */
container: card / inline-size;

/* Multiple containers with different names */
.parent { container: parent / size; }
.child { container: child / inline-size; }
```

2.3 Container Query Syntax

2.3.1 Basic Query

```
/* Query unnamed container */
@container (min-width: 400px) {
  .card { display: grid; }
}

/* Query named container */
@container card (min-width: 400px) {
  .card { display: grid; }
}

/* Multiple conditions */
@container (min-width: 400px) and (min-height: 300px) {
  .card { display: grid; }
}

/* OR condition */
@container (min-width: 500px) or (min-height: 600px) {
  .card { display: grid; }
}
```

Chapter 3

Practical Examples

3.1 Example 1: Responsive Card Component

```
/* Define container */
.card-container {
  container: card / inline-size;
  padding: 16px;
  background: white;
  border-radius: 8px;
}

/* Default: Stack layout */
.card {
  display: block;
}

.card-image {
  width: 100%;
  margin-bottom: 16px;
}

.card-content {
  padding: 0;
}

/* Small container: Column layout */
@container card (min-width: 300px) {
  .card {
    display: flex;
    gap: 16px;
  }

  .card-image {
    width: 100px;
    min-width: 100px;
    height: 100px;
    margin-bottom: 0;
  }
}
```



```
/* Large container: Grid layout */
@container card (min-width: 600px) {
  .card {
    display: grid;
    grid-template-columns: 200px 1fr;
    gap: 24px;
  }

  .card-image {
    width: 200px;
    height: 200px;
  }
}
```

3.2 Example 2: Typography Scaling

```
.text-container {
  container-type: inline-size;
  padding: 20px;
}

/* Base typography */
.title {
  font-size: 16px;
  line-height: 1.3;
}

.description {
  font-size: 14px;
  line-height: 1.5;
}

/* Medium container */
@container (min-width: 400px) {
  .title {
    font-size: 20px;
  }

  .description {
    font-size: 16px;
  }
}

/* Large container */
@container (min-width: 800px) {
  .title {
    font-size: 28px;
  }

  .description {
    font-size: 18px;
  }
}
```

```
}  
}
```

3.3 Example 3: Sidebar Responsive Layout

```
/* Main layout with sidebar */  
.sidebar {  
  container-type: inline-size;  
  width: 300px;  
}  
  
/* When sidebar is narrow */  
@container (max-width: 250px) {  
  .sidebar-item {  
    display: flex;  
    flex-direction: column;  
    text-align: center;  
  }  
  
  .sidebar-icon {  
    width: 100%;  
    margin-bottom: 8px;  
  }  
}  
  
/* When sidebar is wider */  
@container (min-width: 250px) {  
  .sidebar-item {  
    display: flex;  
    align-items: center;  
    gap: 12px;  
  }  
  
  .sidebar-icon {  
    width: 24px;  
    min-width: 24px;  
  }  
}
```

3.4 Example 4: Grid Gallery

```
.gallery-container {  
  container-type: inline-size;  
  display: grid;  
}  
  
/* Single column */  
@container (max-width: 400px) {  
  .gallery {  
    grid-template-columns: 1fr;  
  }  
}
```

```
    }  
  }  
  
  /* Two columns */  
  @container (min-width: 400px) and (max-width: 800px) {  
    .gallery {  
      grid-template-columns: repeat(2, 1fr);  
      gap: 16px;  
    }  
  }  
  
  /* Three columns */  
  @container (min-width: 800px) {  
    .gallery {  
      grid-template-columns: repeat(3, 1fr);  
      gap: 20px;  
    }  
  }  
  
  .gallery-item {  
    aspect-ratio: 1;  
  }
```

3.5 Example 5: Named Multiple Containers

```
/* Parent container */  
.layout {  
  container: layout / size;  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  gap: 20px;  
}  
  
/* Sidebar container */  
.sidebar {  
  container: sidebar / inline-size;  
}  
  
/* Main container */  
.main {  
  container: main / inline-size;  
}  
  
/* Sidebar queries */  
@container sidebar (min-width: 200px) {  
  .sidebar-item { display: flex; }  
}  
  
/* Main content queries */  
@container main (min-width: 600px) {  
  .article { display: grid; }  
}
```

```
/* Layout queries */
@container layout (max-width: 1024px) {
  .layout {
    grid-template-columns: 1fr;
  }
}
```

Chapter 4

Advanced Techniques

4.1 Container Query Units

4.1.1 Container-Relative Units

```
/* Container inline-size units */
.card {
  container-type: inline-size;
  width: 100%;
}

.card-title {
  font-size: 5cqw; /* 5\% of container width */
}

.card-padding {
  padding: 2cqh; /* 2\% of container height */
}

/* Container min unit */
.responsive-width {
  width: 50cqmin; /* 50\% of smaller dimension */
}

/* Container max unit */
.full-height {
  height: 100cqmax; /* 100\% of larger dimension */
}
```

4.2 Nested Containers

```
/* Parent container */
.parent {
  container: parent / inline-size;
}

/* Query parent */
```

```
@container parent (min-width: 800px) {
  .parent { display: grid; }
}

/* Nested child container */
.child {
  container: child / inline-size;
}

/* Query child independently */
@container child (min-width: 300px) {
  .child { display: flex; }
}

/* Can only query immediate container, not ancestors */
@container parent (min-width: 600px) {
  .child { /* Affects only direct children */ }
}
```

4.3 Style-Based Queries

```
/* Query container styles (experimental) */
@container (--dark-mode) {
  .card { background: #333; }
}

/* With container variable */
.container {
  container-type: inline-size;
  --theme: dark;
}

@container (--theme: dark) {
  .card {
    background: #1a1a1a;
    color: white;
  }
}
```

Chapter 5

Browser Support & Fallbacks

5.1 Progressive Enhancement

```
/* Works everywhere */
.card {
    display: block;
}

/* Enhanced: Modern browsers */
@supports (container-type: inline-size) {
    .card-container {
        container-type: inline-size;
    }

    @container (min-width: 400px) {
        .card { display: grid; }
    }
}

/* Fallback: Media queries */
@media (min-width: 400px) {
    .card { display: grid; }
}
```

5.2 Feature Detection

```
/* Check container support */
@supports (container-type: inline-size) {
    /* Use container queries */
}

@supports not (container-type: inline-size) {
    /* Use media queries fallback */
}
```

Chapter 6

Best Practices

6.1 Key Recommendations

6.1.1 1. Use Named Containers

```
/* Good: Clear naming */
.card-container {
  container: card / inline-size;
}

@container card (min-width: 400px) {
  .card { display: grid; }
}

/* Less clear: Unnamed */
.card-container {
  container-type: inline-size;
}

@container (min-width: 400px) {
  .card { display: grid; }
}
```

6.1.2 2. Nest Containers Wisely

```
/* Good: Each level has purpose */
.layout { container: layout / size; }
.sidebar { container: sidebar / inline-size; }
.widget { container: widget / inline-size; }

/* Avoid: Excessive nesting */
.level1 { container-type: inline-size; }
.level2 { container-type: inline-size; }
.level3 { container-type: inline-size; }
.level4 { container-type: inline-size; }
```


6.1.3 3. Test Multiple Contexts

Test components in:

- Narrow containers (200px, 300px)
- Medium containers (400px, 600px)
- Large containers (800px+)
- Different aspect ratios

6.2 Common Patterns

```
/* Single-purpose container */
.component-wrapper {
  container: component / inline-size;
}

/* Size-aware layout */
@container component (min-width: 300px) {
  /* Adapt layout */
}

@container component (min-width: 600px) {
  /* Enhance layout */
}

/* Reusable component */
.card {
  display: block;
}

@container card (min-width: 400px) {
  .card { display: flex; }
}
```

Chapter 7

Common Issues & Solutions

7.1 Issue 1: Container Query Not Triggering

Problem: Styles inside container query don't apply.

Causes:

- Container not defined (missing container-type)
- Size threshold not met
- Incorrect query syntax

Solution:

```
/* Ensure container-type is set */
.container {
  container-type: inline-size; /* Required! */
}

/* Query correct container */
@container (min-width: 400px) {
  .child { display: grid; }
}

/* Debug: Add visible indicator */
@container (min-width: 400px) {
  .child { border: 3px solid red; }
}
```

7.2 Issue 2: Layout Shift

Problem: Content jumps when container size changes.

Solution:

```
/* Use smooth transitions */
.card {
  transition: all 0.3s ease;
}
```

```
@container (min-width: 400px) {  
  .card { display: grid; }  
}  
  
/* Avoid sudden layout changes */  
@container (min-width: 500px) {  
  .card {  
    display: flex;  
    gap: 20px;  
  }  
}
```

Chapter 8

Summary & Quick Reference

8.1 When to Use Container Queries

Use container queries when:

- Building reusable components
- Component layout depends on container
- Different contexts need different layouts
- Creating flexible design systems
- Implementing truly responsive components

8.2 Quick Reference

8.2.1 Common Patterns

```
/* Define container */
container-type: inline-size;

/* Name container */
container: name / inline-size;

/* Query size */
@container (min-width: 400px) { }

/* Query named container */
@container name (min-width: 400px) { }

/* Query both dimensions */
@container (min-width: 300px) and (min-height: 300px) { }
```

8.2.2 Complete Component Example

```
/* Component wrapper */
.card-wrapper {
  container: card / inline-size;
}

/* Base layout */
.card {
  display: block;
}

/* Small: Stack */
@container card (max-width: 399px) {
  .card { flex-direction: column; }
}

/* Medium: Side by side */
@container card (min-width: 400px) {
  .card {
    display: flex;
    gap: 16px;
  }
}

/* Large: Grid */
@container card (min-width: 700px) {
  .card {
    display: grid;
    grid-template-columns: 200px 1fr;
  }
}
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)